

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №2
по дисциплине «Анализ, моделирование и оптимизация систем»
Тема: Моделирование центра массового обслуживания

Студент гр. 1306

Шаврин А.П.

Преподаватель

Мандрикова Б.С.

Санкт-Петербург

2025

Цель работы.

Целью работы является изучение модели обслуживания заявок с неограниченной очередью.

Для достижения поставленной цели требуется решить следующие задачи:

- 1) изучить модель обслуживания заявок со схожими законами распределения потока заявок и обслуживания;
- 2) изучить модель обслуживания заявок с различными законами распределения потока заявок и обслуживания;
- 3) запрограммировать модель;
- 4) сравнить практические результаты с теоретическими.

Постановка задачи.

Вариант 9: $\rho_i = 0,60; 0,85, N = 1500; 47500$

Необходимо смоделировать систему обслуживания заявок с неограниченной очередью с пуассоновским потоком заявок (время отправки сообщения — случайная величина, распределенная по экспоненциальному закону) и тремя различными потоками обслуживания (время обслуживания — случайная величина, распределенная по равномерному, показательному или треугольному закону).

Провести эксперимент и выяснить практические характеристики модели.

Провести теоретический расчет этих параметров.

Оценить результаты.

Выполнение работы.

Пусть интенсивность потока заявок $\lambda = \frac{1}{50} = 0,02$

Для равномерного закона с $\rho = 0,60$

Интенсивность потока обслуживания $\mu = \frac{\lambda}{\rho} = \frac{0,02}{0,60} = 0,03(3)$

Среднее время обслуживания (FT) $\overline{t_{об}} = \frac{1}{\mu} = \frac{0,60}{0,02} = 30$

$$\text{Коэффициент вариации времени обслуживания } \vartheta = \frac{\sigma_{t_{06}}}{\overline{t_{06}}} = \frac{\frac{x}{2\sqrt{3}}}{\frac{x}{2}} = \frac{1}{\sqrt{3}}$$

$$\text{Среднее число заявок в очереди (QA)} \bar{r} = \frac{\rho^2(1+\vartheta^2)}{2(1-\rho)} = \frac{0,60^2*(1+\frac{1}{3})}{2(1-0,60)} = 0.5985$$

$$\text{Среднее время ожидания в очереди (QT)} \overline{t_{ож}} = \frac{\rho^2(1+\vartheta^2)}{2\lambda(1-\rho)} = \frac{0,60^2*(1+\frac{1}{3})}{2*0,02*(1-0,60)} = 29.925$$

Таблица 1 – равномерный закон с $\rho = 0,60$

N	QM	QA	QZ	QT	QX	FR	FT
1500	7	0.564667	1074	28.364533	49.187051	0.589458	29.701268
10 000	8	0.543500	7011	28.113128	47.296649	0.589968	30.027598
47 500	11	0.606295	32541	30.098488	50.157107	0.598941	29.906714
200 000	12	0.603120	137159	30.196259	50.306137	0.599610	30.016070
1 000 000	14	0.596705	688273	29.869590	49.832067	0.598607	29.971242
2 000 000	14	0.600821	1373657	30.04121	50.040451	0.599698	29.996383
Теория		0.5985		29.925			30

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям. При N=2,000,000 практическое значение среднего времени ожидания в очереди (QT) составило 30.04121 единиц времени при теоретическом 29.925. Относительная погрешность составляет всего 0.389%, что указывает на высокую точность моделирования.

Для равномерного закона с $\rho = 0,85$

$$\text{Интенсивность потока обслуживания } \mu = \frac{\lambda}{\rho} = \frac{0,02}{0,85} = 0,0235$$

$$\text{Среднее время обслуживания (FT)} \overline{t_{06}} = \frac{1}{\mu} = \frac{0,85}{0,02} = 42.5$$

$$\text{Коэффициент вариации времени обслуживания } \vartheta = \frac{\sigma_{t_{06}}}{\overline{t_{06}}} = \frac{\frac{x}{2\sqrt{3}}}{\frac{x}{2}} = \frac{1}{\sqrt{3}}$$

$$\text{Среднее число заявок в очереди (QA)} \bar{r} = \frac{\rho^2(1+\vartheta^2)}{2(1-\rho)} = \frac{0,85^2*(1+\frac{1}{3})}{2(1-0,85)} = 3.203$$

$$\text{Среднее время ожидания в очереди (QT)} \overline{t_{ож}} = \frac{\rho^2(1+\vartheta^2)}{2\lambda(1-\rho)} = \frac{0,85^2*(1+\frac{1}{3})}{2*0,02*(1-0,85)} = 160.15$$

Таблица 2 – равномерный закон с $\rho = 0,85$

N	QM	QA	QZ	QT	QX	FR	FT
1750	17	3.312000	469	169.085207	198.923773	0.852084	43.026532
10 000	25	3.199100	3208	160.376373	190.470752	0.846877	42.684266
47 500	41	3.160947	15108	158.279420	186.747620	0.849302	42.445577
200 000	39	3.294795	61302	164.582158	193.001651	0.852725	42.556667
1 000 000	43	3.241939	312457	161.967578	190.594489	0.850021	42.495121
2 000 000	43	3.215918	623949	160.73999	189.110431	0.850015	42.483772
Теория		3.203		160.15			42.5

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям. При $N=2,000,000$ практическое значение среднего времени ожидания в очереди (QT) составило 160.73999 единиц времени при теоретическом 160.15. Относительная погрешность составляет всего 0.3685%, что указывает на высокую точность моделирования.

Для экспоненциального закона с $\rho = 0,60$

Интенсивность потока обслуживания $\mu = \frac{\lambda}{\rho} = \frac{0,02}{0,60} = 0,03(3)$

Среднее время обслуживания (FT) $\overline{t_{об}} = \frac{1}{\mu} = \frac{0,60}{0,02} = 30$

Коэффициент вариации времени обслуживания $\vartheta = \frac{\sigma_{t_{об}}}{\overline{t_{об}}} = \frac{x}{x} = 1$

Среднее число заявок в очереди (QA) $\bar{r} = \frac{\rho^2(1+\vartheta^2)}{2(1-\rho)} = \frac{0,60^2*(1+1)}{2(1-0,60)} = 0.9$

Среднее время ожидания в очереди (QT) $\overline{t_{ож}} = \frac{\rho^2(1+\vartheta^2)}{2\lambda(1-\rho)} = \frac{0,60^2*(1+1)}{2*0,02*(1-0,60)} = 45$

Таблица 3 – экспоненциальный закон с $\rho = 0,60$

N	QM	QA	QZ	QT	QX	FR	FT
1750	8	0.775333	965	41.738172	69.332511	0.596202	31.050559
10 000	16	0.889500	6340	44.328095	73.076318	0.603135	29.884501
47 500	16	0.875074	30767	43.599572	73.405156	0.593701	29.812370
200 000	17	0.887050	128585	44.357770	74.077773	0.5988	29.937565
1 000 000	24	0.896061	640526	44.811903	74.755653	0.600447	30.000343
2 000 000	24	0.903366	1280381	45.218217	75.410072	0.600051	30.009938
Теория		0.9		45			30

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям. При N=2 000 000 практическое значение среднего времени ожидания в очереди (QT) составило 45.218 единиц времени при теоретическом 45.0. Относительная погрешность составляет всего 0.4849%, что указывает на высокую точность моделирования.

Для экспоненциального закона с $\rho = 0,85$

$$\text{Интенсивность потока обслуживания } \mu = \frac{\lambda}{\rho} = \frac{0,02}{0,85} = 0,0235$$

$$\text{Среднее время обслуживания (FT) } \overline{t_{\text{об}}} = \frac{1}{\mu} = \frac{0,85}{0,02} = 42.5$$

$$\text{Коэффициент вариации времени обслуживания } \vartheta = \frac{\sigma_{t_{\text{об}}}}{\overline{t_{\text{об}}}} = \frac{x}{x} = 1$$

$$\text{Среднее число заявок в очереди (QA) } \bar{r} = \frac{\rho^2(1+\vartheta^2)}{2(1-\rho)} = \frac{0,85^2*(1+1)}{2(1-0,85)} = 4.817$$

$$\text{Среднее время ожидания в очереди (QT) } \overline{t_{\text{ож}}} = \frac{\rho^2(1+\vartheta^2)}{2\lambda(1-\rho)} = \frac{0,85^2*(1+1)}{2*0,02*(1-0,85)} =$$

240.83

Таблица 4 – экспоненциальный закон с $\rho = 0,85$

N	QM	QA	QZ	QT	QX	FR	FT
1750	19	3.051333	490	158.322911	191.211246	0.836232	41.722891
10 000	31	4.226500	3021	210.937242	251.955617	0.837458	42.180513
47 500	39	4.767389	12989	236.774902	277.678390	0.852286	42.382566
200 000	43	4.626725	56490	232.363956	273.834880	0.849929	42.521600
1 000 000	70	4.918861	274853	245.477471	288.461053	0.850686	42.524152
3 000 000	695	4.823185	830274	241.205467	283.607313	0.850419	42.505129
Теория		4.817		240.83			42.5

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям. При N=2,000,000 практическое значение среднего времени ожидания в очереди (QT) составило 241.205 единиц времени при теоретическом 240.83. Относительная погрешность составляет всего 0.1560%, что свидетельствует об отличном соответствии теоретической модели.

Для треугольного закона с $\rho = 0,60$

$$\text{Интенсивность потока обслуживания } \mu = \frac{\lambda}{\rho} = \frac{0,02}{0,60} = 0,03(3)$$

$$\text{Среднее время обслуживания (FT)} \overline{t_{об}} = \frac{1}{\mu} = \frac{0,60}{0,02} = 30$$

$$\text{Коэффициент вариации времени обслуживания } \vartheta = \frac{\sigma_{t_{об}}}{\overline{t_{об}}} = \frac{\sqrt{\frac{x^2}{6}}}{x} = \frac{1}{\sqrt{6}}$$

$$\text{Среднее число заявок в очереди (QA)} \bar{r} = \frac{\rho^2(1+\vartheta^2)}{2(1-\rho)} = \frac{0,60^2*(1+\frac{1}{6})}{2(1-0,60)} = 0.525$$

$$\text{Среднее время ожидания в очереди (QT)} \overline{t_{ож}} = \frac{\rho^2(1+\vartheta^2)}{2\lambda(1-\rho)} = \frac{0,60^2*(1+\frac{1}{6})}{2*0,02*(1-0,60)} = 26.25$$

26.25

Таблица 5 – треугольный закон с $\rho = 0,60$

N	QM	QA	QZ	QT	QX	FR	FT
1750	10	0.676000	979	31.956378	51.212144	0.610960	30.300977
10 000	8	0.489800	7186	24.769605	42.283382	0.590881	29.896983
47 500	11	0.524063	33664	26.104803	43.527861	0.596664	29.998696
200 000	13	0.521840	141634	26.088641	43.386702	0.600376	29.942510
1 000 000	13	0.529700	707115	26.398836	44.013905	0.599827	30.016004
2 000 000	14	0.522397	1415538	26.188223	43.663012	0.600184	30.002201
Теория		0.525		26.25			30

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям. При N=2 000 000 практическое значение среднего времени ожидания в очереди (QT) составило 26.188 единиц времени при теоретическом 26.25. Относительная погрешность составляет всего 0.2355%, что свидетельствует об отличном соответствии теоретической модели.

Для треугольного закона с $\rho = 0,85$

$$\text{Интенсивность потока обслуживания } \mu = \frac{\lambda}{\rho} = \frac{0,02}{0,85} = 0,0235$$

$$\text{Среднее время обслуживания (FT)} \overline{t_{об}} = \frac{1}{\mu} = \frac{0,85}{0,02} = 42.5$$

$$\text{Коэффициент вариации времени обслуживания } \vartheta = \frac{\sigma_{t_{об}}}{\overline{t_{об}}} = \frac{\sqrt{\frac{x^2}{6}}}{x} = \frac{1}{\sqrt{6}}$$

$$\text{Среднее число заявок в очереди (QA)} \bar{r} = \frac{\rho^2(1+\vartheta^2)}{2(1-\rho)} = \frac{0,85^2*(1+\frac{1}{6})}{2(1-0,85)} = 2.81$$

$$\text{Среднее время ожидания в очереди (QT)} \quad \overline{t_{\text{ож}}} = \frac{\rho^2(1+\vartheta^2)}{2\lambda(1-\rho)} = \frac{0,85^2 * (1+\frac{1}{6})}{2 * 0,02 * (1-0,85)} =$$

140.486

Таблица 6 – треугольный закон с $\rho = 0,85$

N	QM	QA	QZ	QT	QX	FR	FT
1750	21	3.144667	484	155.664633	182.848042	0.863266	43.132134
10 000	29	3.086000	3076	153.616178	178.581933	0.859971	42.771579
47 500	27	2.980126	15154	148.044820	173.299052	0.854479	42.492018
200 000	31	2.703150	67724	135.709413	160.279451	0.847483	42.412709
1 000 000	41	2.840820	329428	141.956169	166.910057	0.850020	42.518195
3 000 000	44	2.828377	984917	141.304148	166.032081	0.850807	42.509555
Теория		2.81		140.486			42.5

В результате моделирования было получено, что FT, QT, QA близки к теоретическим значениям. При $N=3\,000\,000$ практическое значение среднего времени ожидания в очереди (QT) составило 141.304 единиц времени при теоретическом 140.486. Относительная погрешность составляет 0.5824%, что свидетельствует об отличном соответствии теоретической модели.

Для экспоненциального закона с $\rho = 0,60$ и $N = 100\,000$ проведем 10 экспериментов и рассчитаем среднее время ожидания заявки в очереди и СКО.

Таблица 7. – экспоненциальный закон с $\rho = 0,60$ и $N = 100\,000$

№	1	2	3	4	5	6	7	8	9	10
QT	44.551	43.939	44.438	46.021	44.733	45.735	45.909	44.955	44.734	44.742

Теоретическое время ожидания в очереди: $QT = 45$

Практическое среднее время ожидания в очереди: $QT_{\text{cp}} = 44.976003$

$$\text{СКО: } \sigma(x) = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - x_{\text{cp}})^2} = 0.687668$$

Доверительный интервал при 95%, критическое значение 2.262

$$QT_{\text{cp}} \pm 2.262 \frac{\sigma}{\sqrt{n}} = 44.976003 \pm 0.687668$$

Теоретическое значение соответствует полученному интервалу.

Выводы.

В ходе работы была успешно смоделирована система массового обслуживания с неограниченной очередью и исследовано влияние различных законов распределения времени обслуживания на характеристики системы.

Точность моделирования подтверждена высоким соответствием практических результатов теоретическим расчетам. Для равномерного, экспоненциального и треугольного законов распределения относительная погрешность не превышает 0.5% при $N=3\,000\,000$, что свидетельствует о корректности реализованной модели.

Проведенная серия из 10 экспериментов подтвердила статистическую значимость результатов: теоретическое значение времени ожидания (45.0) попадает в 95% доверительный интервал (44.976 ± 0.688), что свидетельствует о корректности модели и отсутствии систематической погрешности.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

```
from typing import List

import simpy
import numpy as np
import pandas as pd
from enum import Enum, auto
from dataclasses import dataclass

np.random.seed(42)

pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)

class Distribution(Enum):
    UNIFORM = auto()
    EXPONENTIAL = auto()
    TRIANGULAR = auto()

@dataclass
class ExperimentStats:
    # Характеристики очереди
    QM: int # Максимальная длина очереди (Queue Max)
    QA: float # Средняя длина очереди (Queue Average)
    QZ: int # Число заявок, поступивших на обслуживание без
ожидания (Queue Zero)
    QT: float # Среднее время пребывания заявки в очереди
(включая нулевые ожидания)
    QX: float # Среднее время пребывания заявки в очереди
(без нулевых ожиданий)

    # Характеристики устройства (сервера)
```

```

FR: float # Коэффициент загрузки сервера (Facility Rate
/ Utilization)

FT: float # Среднее время обслуживания заявки (Facility
Time / Average Service Time)

```

```

# Дополнительно для анализа вариативности
std_wait_time: float = 0.0 # СКО времени ожидания
std_service_time: float = 0.0 # СКО времени обслуживания

```

```

# Параметры эксперимента
N: int = 0 # Общее число заявок, прошедших через систему
rho: float = 0.0 # Приведенная интенсивность потока ( $\lambda/\mu$ )
service_dist: Distribution = Distribution.UNIFORM

```

```

@dataclass
class UseCase:
    rho: float
    N: int
    service_dist: Distribution

```

```

@dataclass
class UseCaseGroup:
    name: str
    description: str
    use_cases: List[UseCase]

```

```

class TimeGenerator:
    def __init__(self, dist_type: Distribution, avg_time:
float):
        self.__dist_type: Distribution = dist_type
        self.__avg_time: float = avg_time

    def generate(self, count: int = 1) -> float:
        match(self.__dist_type):
            case Distribution.UNIFORM:

```

```

        return np.random.uniform(low=0,
high=self.__avg_time * 2, size=count)
        case Distribution.EXPONENTIAL:
            return
np.random.exponential(scale=self.__avg_time, size=count)
        case Distribution.TRIANGULAR:
            return np.random.triangular(
                left=0,
                mode=self.__avg_time,
                right=self.__avg_time * 2,
                size=count
            )
        case _:
            raise ValueError("Unknown distribution type")

```

```

class ServiceSystem:
    def __init__(self, env, service_gen: TimeGenerator):
        self.env = env
        self.server = simpy.Resource(env, capacity=1)
        self.service_gen = service_gen

        self.wait_times = []
        self.queue_lengths = []
        self.no_wait = 0
        self.service_times = []

    def handle_request(self):
        """Процесс обслуживания заявки"""
        arrival_time = self.env.now
        queue_length = len(self.server.queue)
        self.queue_lengths.append(queue_length)
        if queue_length == 0:
            self.no_wait += 1

        with self.server.request() as req:

```

```

        yield req
        self.wait_times.append(self.env.now
arrival_time)

        service_time = self.service_gen.generate()
        self.service_times.append(service_time)
        yield self.env.timeout(service_time)

class RequestGenerator:
    def __init__(
        self,
        env: simpy.Environment,
        system: ServiceSystem,
        N: int,
        interarrival_gen: TimeGenerator,
    ):
        self.env = env
        self.system = system
        self.N = N
        self.interarrival_gen = interarrival_gen

    def run(self):
        for i in range(self.N):
            interarrival = self.interarrival_gen.generate()
            yield self.env.timeout(interarrival)
            self.env.process(self.system.handle_request())

def run_experiment(N, avg_interval_time, avg_service_time,
service_dist):
    env = simpy.Environment()

    interarrival_gen = TimeGenerator(
        dist_type=Distribution.EXPONENTIAL,
        avg_time=avg_interval_time
    )
    service_gen = TimeGenerator(

```

```

        dist_type=service_dist,
        avg_time=avg_service_time
    )

    system = ServiceSystem(
        env=env,
        service_gen=service_gen
    )

    request_gen = RequestGenerator(
        env=env,
        system=system,
        N=N,
        interarrival_gen=interarrival_gen,
    )

    env.process(request_gen.run())
    env.run()
    total_time: float = env.now

    QX = list(filter(lambda time: time > 0,
system.wait_times))

    return ExperimentStats(
        QM=max(system.queue_lengths),
        QA=np.mean(system.queue_lengths),
        QZ=system.no_wait,
        QT=np.mean(system.wait_times),
        QX=np.mean(QX) if len(QX) else 0.0,
        FR=sum(system.service_times) / total_time,
        FT=np.mean(system.service_times),
    )

def run_experiments(N: int, rho: float, lam: float,
service_dist: Distribution, repeats: int) -> ExperimentStats:

```

```

mu: float = lam / rho
avg_service_time: float = 1 / mu
avg_interval_time: float = 1 / lam

all_stats: list[ExperimentStats] = []
for _ in range(repeats):
    experiment_statistic = run_experiment(
        N=N,
        avg_interval_time=avg_interval_time,
        avg_service_time=avg_service_time,
        service_dist=service_dist
    )
    all_stats.append(experiment_statistic)

return ExperimentStats(
    N=N,
    rho=rho,
    service_dist=service_dist,
    QM=int(np.mean(list(map(lambda stat: stat.QM,
all_stats))))),
    QA=float(np.mean(list(map(lambda stat: stat.QA,
all_stats))))),
    QZ=int(np.mean(list(map(lambda stat: stat.QZ,
all_stats))))),
    QT=float(np.mean(list(map(lambda stat: stat.QT,
all_stats))))),
    QX=float(np.mean(list(map(lambda stat: stat.QX,
all_stats))))),
    FR=float(np.mean(list(map(lambda stat: stat.FR,
all_stats))))),
    FT=float(np.mean(list(map(lambda stat: stat.FT,
all_stats))))),
    std_wait_time=float(np.std(list(map(lambda stat:
stat.QT, all_stats))), ddof=1)),

```

```

        std_service_time=float(np.std(list(map(lambda stat:
stat.FT, all_stats))), ddof=1))
    )

def main():
    lam: float = 1 / 50
    use_cases_groups = [
        UseCaseGroup(
            name="Равномерное распределение  $\rho=0.6$ ",
            description="Исследование для равномерного закона
обслуживания с низкой нагрузкой",
            use_cases=[
                UseCase(rho=0.6, N=1500,
service_dist=Distribution.UNIFORM),
                UseCase(rho=0.6, N=10_000,
service_dist=Distribution.UNIFORM),
                UseCase(rho=0.6, N=47_500,
service_dist=Distribution.UNIFORM),
                UseCase(rho=0.6, N=200_000,
service_dist=Distribution.UNIFORM),
                UseCase(rho=0.6, N=1_000_000,
service_dist=Distribution.UNIFORM),
                UseCase(rho=0.6, N=2_000_000,
service_dist=Distribution.UNIFORM),
            ]
        ),
        UseCaseGroup(
            name="Равномерное распределение  $\rho=0.85$ ",
            description="Исследование для равномерного закона
обслуживания с высокой нагрузкой",
            use_cases=[
                UseCase(rho=0.85, N=1500,
service_dist=Distribution.UNIFORM),
                UseCase(rho=0.85, N=10_000,
service_dist=Distribution.UNIFORM),
            ]
        )
    ]

```

```

        UseCase(rho=0.85, N=47_500,
service_dist=Distribution.UNIFORM),
        UseCase(rho=0.85, N=200_000,
service_dist=Distribution.UNIFORM),
        UseCase(rho=0.85, N=1_000_000,
service_dist=Distribution.UNIFORM),
        UseCase(rho=0.85, N=2_000_000,
service_dist=Distribution.UNIFORM),
    ]
),
    UseCaseGroup(
        name="Экспоненциальное распределение  $\rho=0.6$ ",
        description="Исследование для экспоненциального
закона обслуживания с низкой нагрузкой",
        use_cases=[
            UseCase(rho=0.6, N=1500,
service_dist=Distribution.EXPONENTIAL),
            UseCase(rho=0.6, N=10_000,
service_dist=Distribution.EXPONENTIAL),
            UseCase(rho=0.6, N=47_500,
service_dist=Distribution.EXPONENTIAL),
            UseCase(rho=0.6, N=200_000,
service_dist=Distribution.EXPONENTIAL),
            UseCase(rho=0.6, N=1_000_000,
service_dist=Distribution.EXPONENTIAL),
            UseCase(rho=0.6, N=2_000_000,
service_dist=Distribution.EXPONENTIAL),
        ]
    ),
    UseCaseGroup(
        name="Экспоненциальное распределение  $\rho=0.85$ ",
        description="Исследование для экспоненциального
закона обслуживания с высокой нагрузкой",
        use_cases=[

```



```

        UseCase(rho=0.85, N=1500,
service_dist=Distribution.EXPONENTIAL),
        UseCase(rho=0.85, N=10_000,
service_dist=Distribution.EXPONENTIAL),
        UseCase(rho=0.85, N=47_500,
service_dist=Distribution.EXPONENTIAL),
        UseCase(rho=0.85, N=200_000,
service_dist=Distribution.EXPONENTIAL),
        UseCase(rho=0.85, N=1_000_000,
service_dist=Distribution.EXPONENTIAL),
        UseCase(rho=0.85, N=2_000_000,
service_dist=Distribution.EXPONENTIAL),
    ]
),
    UseCaseGroup(
        name="Треугольное распределение  $\rho=0.6$ ",
        description="Исследование для треугольного закона
обслуживания с низкой нагрузкой",
        use_cases=[
            UseCase(rho=0.6, N=1500,
service_dist=Distribution.TRIANGULAR),
            UseCase(rho=0.6, N=10_000,
service_dist=Distribution.TRIANGULAR),
            UseCase(rho=0.6, N=47_500,
service_dist=Distribution.TRIANGULAR),
            UseCase(rho=0.6, N=200_000,
service_dist=Distribution.TRIANGULAR),
            UseCase(rho=0.6, N=1_000_000,
service_dist=Distribution.TRIANGULAR),
            UseCase(rho=0.6, N=2_000_000,
service_dist=Distribution.TRIANGULAR),
        ]
    ),
    UseCaseGroup(
        name="Треугольное распределение  $\rho=0.85$ ",

```

```

        description="Исследование для треугольного закона
обслуживания с высокой нагрузкой",
        use_cases=[
            UseCase(rho=0.85, N=1500,
service_dist=Distribution.TRIANGULAR),
            UseCase(rho=0.85, N=10_000,
service_dist=Distribution.TRIANGULAR),
            UseCase(rho=0.85, N=47_500,
service_dist=Distribution.TRIANGULAR),
            UseCase(rho=0.85, N=200_000,
service_dist=Distribution.TRIANGULAR),
            UseCase(rho=0.85, N=1_000_000,
service_dist=Distribution.TRIANGULAR),
            UseCase(rho=0.85, N=3_000_000,
service_dist=Distribution.TRIANGULAR),
        ]
    ),
]

for group in use_cases_groups:
    print(f"\n{'=' * 60}")
    print(f"ГРУППА: {group.name}")

    group_results = []
    for use_case in group.use_cases:
        stats = run_experiments(
            N=use_case.N,
            rho=use_case.rho,
            lam=lam,
            service_dist=use_case.service_dist,
            repeats=1
        )
        group_results.append(stats)

```

```
        df_group_results = pd.DataFrame([s.__dict__ for s in
group_results])
        print(df_group_results)

if __name__ == "__main__":
    main()
```